

KONSPEKT LABORATORIUM 8

Implementacja dostępności – ARIA i semantyczny HTML

Czas trwania: 90 minut • Rok I, Semestr I • Informatyka II stopnia

Prowadzący

mgr inż. Dominik Aksamit

Numer lab.

Laboratorium 8 z 15

1. Cele zajęć

1.1 Cele ogólne

Po zakończeniu laboratorium student będzie potrafił przeprowadzić kompleksową refaktoryzację aplikacji webowej w celu spełnienia wymogów dostępności zgodnych z WCAG 2.1 poziom AA.

1.2 Szczegółowe efekty kształcenia

Student, który zaliczył zajęcia, w zakresie —

WIEDZY:

- zna strukturę semantycznych elementów HTML5 i ich wpływ na drzewko dostępności (accessibility tree),
- rozumie hierarchię ról ARIA i zasadę „ARIA in HTML” (pierwszeństwo semantyki natywnej),
- zna specyfikację WAI-ARIA Authoring Practices Guide (APG) dla wzorców: modal, navigation, live region.

UMIEJĘTNOŚCI:

- potrafi zastąpić elementy `<div>/<span>` semantycznymi elementami HTML5 bez zerwania stylowania CSS,
- implementuje skip navigation link, ARIA live region i focus trap w React,
- weryfikuje dostępność aplikacji narzędziami Lighthouse / axe-core oraz nawigacją klawiaturową.

KOMPETENCJE SPOŁECZNYCH:

- rozumie znaczenie inkluzywności i etyki projektowania dla osób z niepełnosprawnościami (efekt uczenia IN2_K01, IN2_K02).

2. Opis zadania

Zadanie polega na przeprowadzeniu refaktoryzacji aplikacji zrealizowanej na poprzednim laboratorium (Lab 7). Refaktoryzacja obejmuje pięć obszarów, które opisano poniżej.

2.1 Zakres refaktoryzacji

#	Obszar	Oczekiwany rezultat	Kryterium WCAG
A	Semantyczny HTML5	Zastąpienie <div> elementami: main, nav, article, section, header, footer	1.3.1 (A)
B	Role i atrybuty ARIA	Dodanie aria-label, aria-labelledby, aria-describedby tam, gdzie semantyka HTML5 jest niewystarczająca	4.1.2 (A)
C	Skip navigation	Widoczny link „Przejdź do treści” pojawiający się przy fokusie	2.4.1 (A)
D	ARIA live region	Region status dla wyników wyszukiwania odczytywany przez czytniki ekranu	4.1.3 (AA)
E	Focus trap w modalu	Klawiatura uwięziona w modalu; wyjście Escape zamyka okno i przywraca fokus	2.1.2 (A)

3. Instrukcja krok po kroku

Krok 1 — Semantyczny HTML5 (15 min)

Przejdź przez cały plik JSX i zamień każdy element <div> pełniący rolę obszaru na odpowiedni element HTML5:

Przed refaktoryzacją

Typowa struktura przed zmianą — same <div> bez semantycznego znaczenia:

```
// ✗ PRZED — brak semantyki
<div className="app">
  <div className="header">
    <div className="nav">
      <div className="nav-link">Strona główna</div>
    </div>
  </div>
  <div className="main-content">
    <div className="article">...</div>
    <div className="aside">...</div>
  </div>
  <div className="footer">...</div>
</div>
```

Po refaktoryzacji

Elementy HTML5 niosą semantyczne znaczenie — czytniki ekranu rozumieją strukturę strony:

```
// ☑ PO — semantyczny HTML5
<div className="app">
  <header className="header">
    <nav aria-label="Nawigacja główna">
      <a href="/">Strona główna</a>
```

</nav>
</header>
<main id="main-content">
<article>...</article>
<aside aria-label="Informacje dodatkowe">...</aside>
</main>
<footer>
© 2025 Akademia Tarnowska
</footer>
</div>

Krok 2 — Role ARIA i atrybuty (15 min)

Dodaj atrybuty ARIA do elementów, które nie mają wystarczającej semantyki natywnej:

// Przykład: formularz wyszukiwania
<form role="search" aria-label="Wyszukiwarka produktów">
<label htmlFor="search-input">
Wyszukaj produkt
</label>
<input
id="search-input"
type="search"
aria-describedby="search-hint"
aria-autocomplete="list"
aria-controls="search-results"
/>

Wpisz co najmniej 3 znaki

<button type="submit" aria-label="Szukaj">
<SearchIcon aria-hidden="true" />
</button>
</form>
/* CSS – klasa ukrywająca wizualnie, lecz dostępna dla czytników */
.visually-hidden {
position: absolute;
width: 1px; height: 1px;
padding: 0; margin: -1px;
overflow: hidden;
clip: rect(0,0,0,0);
white-space: nowrap;
border: 0;
}

Krok 3 — Skip navigation link (10 min)

Umieść skip link jako pierwszy element <body>. Link powinien być widoczny dopiero po otrzymaniu fokusu klawiaturowego:

```
// App.jsx – pierwszy element JSX
function App() {
  return (
    <>
      <a
        href="#main-content"
        className="skip-link"
      >
        Przejdź do treści głównej
      </a>
      <header>...</header>
      <main id="main-content" tabIndex={-1}>
        ...
      </main>
    </>
  );
}

/* CSS – skip link */
.skip-link {
  position: absolute;
  top: -40px;
  left: 0;
  background: #0043FF;
  color: white;
  padding: 8px 16px;
  z-index: 9999;
  font-weight: bold;
  transition: top 0.2s;
}
.skip-link:focus {
  top: 0;
}
```

Krok 4 — ARIA live region (15 min)

Live region informuje czytnik ekranu o dynamicznych zmianach treści bez konieczności przesuwania fokusu:

```
// SearchResults.jsx
function SearchResults({ results, query, isLoading }) {
  return (
    <section aria-label="Wyniki wyszukiwania">
      {/* ARIA live region – ogłasza zmiany czytnikowi ekranu */}
      <div
```

role="status"
aria-live="polite"
aria-atomic="true"
className="visually-hidden"
>
{isLoading && "Wyszukiwanie..."}
{!isLoading && results.length === 0 &&
`Brak wyników dla zapytania: \${query}`}
{!isLoading && results.length > 0 &&
`Znaleziono \${results.length} wyników dla: \${query}`}
</div>
{/* Wizualna lista wyników */}
<ul id="search-results">
{results.map(item => (
<li key={item.id}>
{item.title}
)}}
</section>
);
}

Wskazówka — aria-live vs role="status"

Wartość polite poczeka z odczytem do końca aktualnej wypowiedzi czytnika. Użyj assertive tylko dla błędów krytycznych (np. błąd autoryzacji). role="status" implikuje aria-live="polite" — użyć jednego lub drugiego, nie obu.

Krok 5 — Focus trap w modalu (20 min)

Komponent FocusTrap przechwytuje nawigację Tab/Shift+Tab wewnątrz modala. Szczegółowy kod gotowego komponentu znajduje się w rozdziale 4.

// ModalDialog.jsx – użycie komponentu FocusTrap
import { FocusTrap } from './FocusTrap';
function ModalDialog({ isOpen, onClose, title, children }) {
const triggerRef = useRef(null); // referencja do elementu otwierającego
if (!isOpen) return null;
return (
<div
className="modal-overlay"
onClick={onClose}
aria-hidden="true"

```

    >
    <FocusTrap onEscape={onClose} triggerRef={triggerRef}>
      <div
        role="dialog"
        aria-modal="true"
        aria-labelledby="modal-title"
        className="modal-content"
        onClick={e => e.stopPropagation()}
      >
        <h2 id="modal-title">{title}</h2>
        {children}
        <button onClick={onClose}>Zamknij</button>
      </div>
    </FocusTrap>
  </div>
);
}

```

4. Gotowy komponent FocusTrap

Poniższy komponent jest gotowy do wielokrotnego użycia. Skopiuj go do pliku `src/components/FocusTrap.jsx`:

```

// src/components/FocusTrap.jsx
import { useEffect, useRef } from 'react';

const FOCUSABLE_SELECTORS = [
  'a[href]',
  'button:not([disabled])',
  'input:not([disabled])',
  'select:not([disabled])',
  'textarea:not([disabled])',
  '[tabindex]:not([tabindex="-1"])',
  'details > summary',
].join(', ');

/**
 * FocusTrap – utrzymuje fokus klawiaturowy wewnątrz kontenera.
 *
 * @param {React.ReactNode} children – zawartość pułapki
 * @param {Function} onEscape – funkcja wywoływana przy naciśnięciu
 * Escape
 * @param {React.RefObject} triggerRef – element, który otworzył pułapkę
 */
export function FocusTrap({ children, onEscape, triggerRef }) {
  const containerRef = useRef(null);

  useEffect(() => {

```

```
const container = containerRef.current;
if (!container) return;

// Pobierz wszystkie elementy z możliwością fokusu
const getFocusable = () =>
  Array.from(container.querySelectorAll(FOCUSABLE_SELECTORS))
    .filter(el => !el.closest('[aria-hidden="true"]'));

// Ustaw fokus na pierwszym elemencie po podłączeniu komponentu
const firstFocusable = getFocusable()[0];
firstFocusable?.focus();

function handleKeyDown(e) {
  const focusable = getFocusable();
  const first = focusable[0];
  const last = focusable[focusable.length - 1];

  if (e.key === 'Escape') {
    onEscape?.();
    // Przywraca fokus do elementu, który otworzył pułapkę
    triggerRef?.current?.focus();
    return;
  }

  if (e.key !== 'Tab') return;

  if (e.shiftKey) {
    // Shift+Tab: jeśli na pierwszym – przejdź na ostatni
    if (document.activeElement === first) {
      e.preventDefault();
      last?.focus();
    }
  } else {
    // Tab: jeśli na ostatnim – przejdź na pierwszy
    if (document.activeElement === last) {
      e.preventDefault();
      first?.focus();
    }
  }
}

document.addEventListener('keydown', handleKeyDown);

return () => {
  document.removeEventListener('keydown', handleKeyDown);
  // Przywróć fokus przy odłączeniu komponentu
  triggerRef?.current?.focus();
};
```

```

    }, [onEscape, triggerRef]);

    return <div ref={containerRef}>{children}</div>;
  }

```

Jak działa FocusTrap?

1) Po podłączeniu komponent wybiera wszystkie elementy z możliwością fokusu (selektor CSS powyżej). 2) Ustawia fokus na pierwszym z nich. 3) Nasłuchuje na Tab / Shift+Tab i zapobiega wyjściu poza kontener. 4) Escape wywołuje onEscape i przywraca fokus do elementu wyzwalającego. 5) Przy odłączeniu komponentu fokus jest zawsze zwracany.

5. Kiedy ARIA, a kiedy natywny HTML?

Pierwsza zasada ARIA (WAI-ARIA 1.2, §1.1): „nie używaj roli ARIA, jeśli możesz użyć natywnego elementu HTML z wbudowaną semantyką”.

Scenariusz	Natywny HTML ☒	ARIA (gdy HTML niewystarczający) ⚠	WCAG
Przycisk akcji	<code><button>Zapisz</button></code>	<code>role="button"</code> (gdy <code><div></code> musi działać jak przycisk)	4.1.2
Formularz	<code><form><label>...</label></form></code>	<code>aria-label</code> na <code><div role="form"></code>	1.3.1
Nawigacja	<code><nav aria-label="..."></code>	<code>role="navigation"</code> na <code><div></code> (unikąć)	4.1.2
Nagłówek	<code><h1></code> – <code><h6></code>	<code>role="heading" aria-level="2"</code> (wady: nie aktualizuje konspektu)	1.3.1
Lista	<code>/</code> + <code></code>	<code>role="list" + role="listitem"</code> (jeśli CSS <code>list-style: none</code>)	1.3.1
Dialog modalny	Brak natywnego – używaj ARIA	<code>role="dialog" aria-modal aria-labelledby</code>	4.1.2
Alert błędu	<code><output></code> lub <code><p></code>	<code>role="alert"</code> (wymusza natychmiastowy odczyt)	4.1.3
Zakładki (tabs)	Brak natywnego	<code>role="tablist" role="tab" role="tabpanel"</code>	4.1.2
Postęp zadania	<code><progress value="70" max="100"></code>	<code>role="progressbar" aria-valuenow aria-valuemax</code>	4.1.2
Stan rozwijany	<code><details><summary></code>	<code>aria-expanded aria-controls</code> (dla niestandardowych)	4.1.2

6. Test weryfikacyjny — nawigacja klawiaturowa

Przejdź przez całą aplikację używając TYLKO klawiatury. Nie dotykaj myszy ani touchpada. Odznacz każdy punkt po pozytywnej weryfikacji.

Jak przeprowadzić test?

1) Wyłącz mysz / touchpad. 2) Otwórz aplikację w Chrome lub Firefox. 3) Naciśnij Tab — powinien pojawić się skip link. 4) Przejdź przez wszystkie elementy. 5) Równolegle możesz uruchomić NVDA (Windows) lub VoiceOver (Mac) i słuchać odczytów.

6.1 Skip Link

- ❑ Po naciśnięciu Tab (pierwsza interakcja) pojawia się widoczny link „Przejdź do treści głównej”.
- ❑ Naciśnięcie Enter na skip linku przenosi fokus do elementu `<main id="main-content">`.
- ❑ Skip link znika (lub jest niewidoczny) po utracie fokusu.

6.2 Nawigacja po stronie

- ❑ Kolejność Tab odpowiada logicznej kolejności wizualnej (od lewej do prawej, od góry do dołu).
- ❑ Każdy element interaktywny (przycisk, link, pole formularza) jest osiągalny klawiaturą.
- ❑ Fokus jest zawsze widoczny (widoczny kontur / outline) — brak elementów z `outline: none` bez zamiennika.
- ❑ Ikon graficzne bez tekstu posiadają `aria-label` lub `aria-hidden="true"` z widocznym tekstem obok.

6.3 Formularz wyszukiwania

- ❑ Pole wyszukiwania jest poprawnie oznakowane etykietą (`<label>` powiązane przez `for/htmlFor`).
- ❑ Po wpisaniu zapytania i naciśnięciu Enter lub kliknięciu przycisku — czytnik ekranu ogłasza wyniki.
- ❑ Komunikat „Znaleziono X wyników” jest ogłaszany przez live region (można zweryfikować w Accessibility Tree w DevTools).

6.4 Modal / dialog

- ❑ Otwarcie modala (np. Enter/Space na przycisku) przenosi fokus do pierwszego elementu wewnątrz.
- ❑ Tab / Shift+Tab krąży wyłącznie wewnątrz modala — nie wycieka na tło strony.
- ❑ Naciśnięcie Escape zamyka modal.
- ❑ Po zamknięciu modala fokus wraca do elementu, który go otworzył.

6.5 Test narzędziami automatycznymi

- ❑ Lighthouse (Accessibility) ≥ 90 pkt.
- ❑ axe DevTools: 0 błędów krytycznych (critical) i poważnych (serious).
- ❑ W Accessibility Tree (Chrome DevTools > Elements > Accessibility) każdy element interaktywny ma Name, Role i State.

7. Kryteria oceny

Ocena końcowa wystawiana jest na podstawie średniej ważonej zgodnie z Regulaminem Studiów Akademii Tarnowskiej.

#	Kryterium	Waga	Opis weryfikacji
---	-----------	------	------------------

1	Semantyczny HTML5	20%	Poprawne użycie co najmniej 5 elementów semantycznych; brak zbędnych <div> pełniących rolę semantyczną.
2	Role ARIA i atrybuty	20%	Atrybuty aria-label / aria-labelledby / aria-describedby zastosowane poprawnie; brak nadmiarowych ról powielających semantykę HTML.
3	Skip navigation link	15%	Link widoczny przy fokusie, przenosi do <main>, nie powoduje zerwania przepływu Tab.
4	ARIA live region	20%	Region ogłasza wyniki wyszukiwania; użyto aria-live="polite" lub role="status"; zawartość jest aktualizowana dynamicznie.
5	Focus trap w modalu	25%	Fokus nie wycieka z modala; Escape zamyka i przywraca fokus; Shift+Tab działa poprawnie.

Literatura i zasoby

- MDN Web Docs — ARIA: <https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA>
- WAI-ARIA Authoring Practices Guide 1.2: <https://www.w3.org/WAI/ARIA/apg/>
- Web Content Accessibility Guidelines (WCAG) 2.1: <https://www.w3.org/TR/WCAG21/>
- axe-core — biblioteka do automatycznego testowania dostępności: <https://github.com/dequelabs/axe-core>